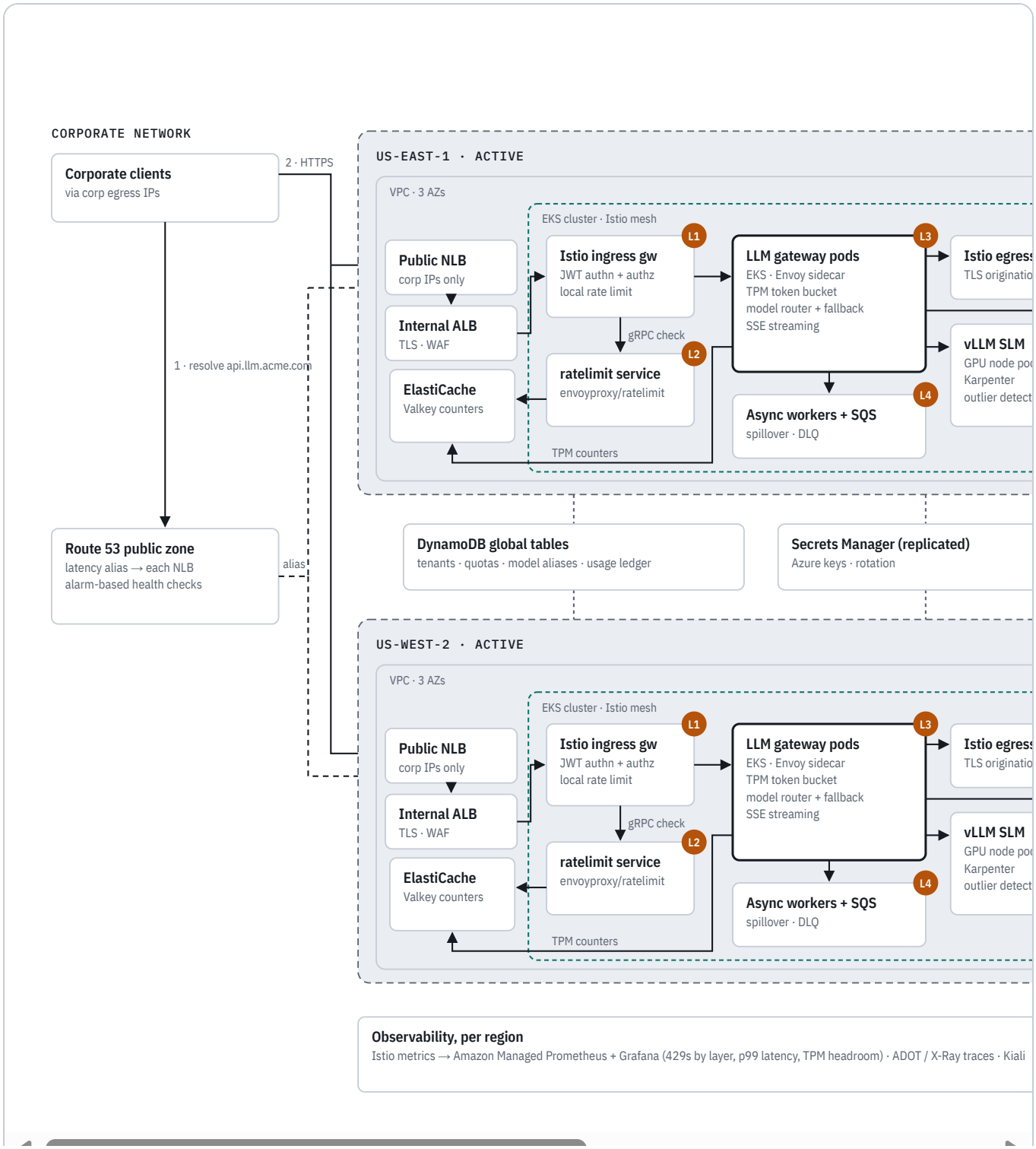
REFERENCE ARCHITECTURE · CORPORATE-ONLY · AWS + ISTIO

Multi-Region LLM Gateway

One OpenAI-compatible API for corporate teams in front of Amazon Bedrock, Azure OpenAI and a self-hosted SLM. A public NLB admits only corporate egress IPs and hands traffic to an internal ALB with TLS and AWS WAF. Istio handles authentication, request rate limits and egress in two active AWS regions, and the gateway enforces token budgets and falls back between providers.





L1 rate-limit layer, in the order a request meets them the service you build and own

Istio mesh boundary outside AWS DNS answer (alias record)

replicated across regions

Both regions serve traffic. The NLB is the only public piece, and its security group admits only corporate egress IPs. Behind it, the internal ALB terminates TLS and runs WAF, and Istio's ingress gateway is the single door into the mesh. It authenticates the caller and enforces request limits before a gateway pod does any work, and all traffic to Azure leaves through one egress gateway.

Request path

1. The client resolves `api.llm.acme.com` in the public Route 53 zone. Its latency alias records point at each region's public NLB; a region whose health check fails drops out of the answer.
2. The client connects over HTTPS from a corporate egress IP. The NLB's security group refuses every other source and passes the connection to the internal ALB, which terminates TLS (ACM certificate), applies AWS WAF and re-encrypts to the Istio ingress gateway.
3. The ingress gateway validates the JWT (`RequestAuthentication`) and checks allowed models (`AuthorizationPolicy`). It copies the tenant claim into `x-tenant-id` and applies the per-pod local limit (**L1**).
4. The ingress gateway asks the ratelimit service for the tenant × model request budget (**L2**). Over budget returns 429 before any gateway pod runs.
5. A gateway pod reserves tokens (prompt + `max_tokens`) in Valkey (**L3**). It then routes the model alias to the first provider with headroom (**L4**): Bedrock over PrivateLink, vLLM in-mesh, or Azure through the egress gateway.
6. The response streams back over SSE. Actual usage reconciles the bucket and lands in the DynamoDB ledger and the S3 audit log.

Rate-limit layers

LAYER	WHERE	KEYED ON	WHEN EXCEEDED
L1 · local	Envoy `local_ratelimit` on each ingress pod	per pod, optionally per source	429 immediately, no network hop. Protects pods from bursts.
L2 · global	Envoy `ratelimit` filter → ratelimit service → Valkey	tenant × model descriptors	429 with `x-envoy-ratelimited` . Shared across every ingress pod in the region.
L3 · tokens	Gateway pods, Valkey Lua script	tenant × model, tokens per minute	429 with `Retry-After` and `x-ratelimit-*` headers

L4 · provider	Router headroom + Istio `DestinationRule` pools	provider × deployment	Next provider in chain, then SQS for async tenants, then 503

Envoy's rate limiter counts requests, not tokens, so token budgets stay in the gateway pods. If you want tokens enforced in the proxy too, Envoy AI Gateway can read usage from the response, but it adds another control plane. Each region holds its own L2 and L3 counters. A tenant's global quota is split across regions and rebalanced from the DynamoDB ledger.

Failure handling

FAILURE	DETECTED BY	RESPONSE
Region down	CloudWatch alarm-based Route 53 health check	The public zone answers with the other region within 1–2 minutes. Each region is sized for full peak.
AZ or ingress pod lost	NLB target health	Ingress pods in other AZs take over. HPA and a PodDisruptionBudget keep capacity.
ratelimit service down	gRPC errors at Envoy	`failure_mode_deny: false` fails open. L1 still caps each pod, and an alarm fires.
Valkey down	Connection errors	L2 fails open as above. Gateway pods fall back to a conservative in-memory token bucket.
Bedrock 429 / 5xx	App circuit breaker	Cross-region inference profile first, then the Azure equivalent of the alias
Azure outage	Outlier detection on the egress path	Second Azure region, then Bedrock
GPU saturated	Envoy pool overflow (503), vLLM queue depth	Router falls back to Bedrock or SQS while Karpenter adds nodes

Istio configuration

Trimmed starting points for the ingress gateway in `istio-system`. Header names, limits and hostnames are examples to replace.

L1 · local rate limit

Token bucket inside each ingress Envoy. Cheap, and it stays up even when everything else fails.

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
```

```
  name: ingress-local-ratelimit
  namespace: istio-system
spec:
  workloadSelector:
    labels:
      istio: ingressgateway
  configPatches:
  - applyTo: HTTP_FILTER
    match:
      context: GATEWAY
      listener:
        filterChain:
          filter:
            name: envoy.filters.network.http_connection_manager
    patch:
      operation: INSERT_BEFORE
      value:
        name: envoy.filters.http.local_ratelimit
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.Loca
          stat_prefix: http_local_rate_limiter
          token_bucket:
            max_tokens: 500
            tokens_per_fill: 500
            fill_interval: 1s
          filter_enabled:
            runtime_key: local_rate_limit_enabled
            default_value: {numerator: 100, denominator: HUNDRED}
          filter_enforced:
            runtime_key: local_rate_limit_enforced
            default_value: {numerator: 100, denominator: HUNDRED}
```

L2 · global rate limit

Calls the ratelimit service before routing, and builds the descriptor from the tenant and model headers.

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: ingress-global-ratelimit
  namespace: istio-system
spec:
  workloadSelector:
    labels:
      istio: ingressgateway
  configPatches:
  - applyTo: HTTP_FILTER
    match:
      context: GATEWAY
      listener:
```

```
      filterChain:
        filter:
          name: envoy.filters.network.http_connection_manager
          subFilter:
            name: envoy.filters.http.router
  patch:
    operation: INSERT_BEFORE
    value:
      name: envoy.filters.http.ratelimit
      typed_config:
        "@type": type.googleapis.com/envoy.extensions.filters.http.ratelimit.v3.RateLimit
        domain: llm-gateway
        failure_mode_deny: false
        timeout: 25ms
        rate_limit_service:
          transport_api_version: V3
          grpc_service:
            envoy_grpc:
              cluster_name: outbound|8081||ratelimit.ratelimit.svc.cluster.local
              authority: ratelimit.ratelimit.svc.cluster.local
- applyTo: VIRTUAL_HOST
  match:
    context: GATEWAY
    routeConfiguration:
      vhost:
        name: "api.llm.acme.com:443"
        route:
          action: ANY
  patch:
    operation: MERGE
    value:
      rate_limits:
      - actions:
        - request_headers:
            header_name: x-tenant-id
            descriptor_key: tenant
        - request_headers:
            header_name: x-model-alias
            descriptor_key: model
```

ratelimit service rules

The ConfigMap for `envoyproxy/ratelimit`. A rule with a specific value overrides the default for that tenant.